
Numerical Domains and Abstract Interpretation: An Informal Introduction

R. BAGNARA

Dipartimento di Scienze Matematiche, Fisiche e
Informatiche
Università di Parma

`roberto.bagnara@unipr.it`

PLAN OF THE TALK

- ① The General Problem
- ② Examples of Verification Problems
- ③ Formal Program Verification Methods
- ④ Abstract Interpretation by Example
- ⑤ Numeric Abstract Domains
- ⑥ A Taste of Concrete and Abstract Semantics
- ⑦ Little Demo

THE PROBLEM

- Hardware is **millions of times more powerful** than it was 25 years ago;
- program sizes have **exploded** in similar proportions;
- large and very large programs (up to **tens of millions of lines of code**) are and will be in widespread use;
- they need to be designed, developed and maintained over their entire lifespan (up to 20 and more years) at **reasonable costs**;
- unassisted development and maintenance teams do not stand a chance to follow such an explosion in size and complexity;
- many pieces of software exhibit a number of bugs that is sometimes hardly bearable even in office applications...
 - ...no **safety critical** application can tolerate this failure rate;
- the problem of **software reliability** is one of the most important problems computer science has to face;
- this justifies the growing interest in **mechanical tools to help the programmer reasoning about programs**.

ARIANE 5: CONSEQUENCES OF `y := int(x)`



“[...] a program segment for converting a floating point number to a signed 16 bit integer was executed with an input data value outside the range representable by a signed 16 bit integer. This run time error (out of range, **overflow**), which arose in both the active and the backup computers at about the same time, was detected and both computers shut

themselves down. This resulted in the total loss of attitude control. The Ariane 5 turned uncontrollably and aerodynamic forces broke the vehicle apart. This breakup was detected by an on-board monitor which ignited the explosive charges to destroy the vehicle in the air. Ironically, the result of this format conversion was no longer needed after lift off.”

HISTORICAL DISASTERS I

[...] The Mars Climate Orbiter **burned** in the martian atmosphere in 1999 after missing its orbit insertion **because unit computations were inconsistent**.

The same year, Mars Polar Lander is suspected of having **crashed** on Mars upon landing **when a software flag was not reset properly**.

In [...] the 1997 Mars Pathfinder (MPF) technology demonstration mission [...] a day's **exploration time was lost when ground support teams were forced to reboot the system** while downloading science data.

[...]

NASA's 2003 Mars Exploration Rover (MER) mission includes two rovers

[...] At a cost of \$400 million for each rover, **a coding error that shuts down a rover overnight would in effect be a \$4.4 million mistake**, as well as a loss of valuable exploration time on the planet.

<http://www.arc.nasa.gov/exploringtheuniverse-computercheck.cfm>

HISTORICAL DISASTERS II

A previously-unknown **software flaw** in a widely-deployed General Electric energy management system contributed to the devastating scope of the August 14th northeastern U.S. blackout, industry officials revealed this week.

The bug in GE Energy's XA/21 system was discovered in an intensive code audit conducted by GE and a contractor in the weeks following the blackout, according to FirstEnergy Corp., the Ohio utility where investigators say the blackout began. **"It had never evidenced itself until that day,"** said spokesman Ralph DiNicola. **"This fault was so deeply embedded, it took them weeks of poring through millions of lines of code and data to find it."**

[...]

The cascading blackout eventually cut off electricity to 50 million people in eight states and Canada.

<http://www.securityfocus.com/news/8016>

HISTORICAL DISASTERS III

On July 1, 2002, a Russian Tupolev TU154M passenger airplane found itself on a collision course with a Boeing B757-200 cargo plane. The Traffic Alert and Collision Avoidance System (TCAS), correctly alerted the pilot to avoid the collision by **pulling the plane up**, while the human airspace controller ordered it **to push down**. The pilot obeyed the human, which resulted in a collision that killed 71 passengers and crew.

ICAO [International Civil Aviation Organization] should change the international requirements [...] so that pilots flying are required to obey and **follow TCAS resolution advisories (RAs), regardless of whether contrary ATC [Air Traffic Control] instruction is given prior to, during, or after the RAs are issued**. Unless the situation is too dangerous to comply, the pilot flying should comply with the RA until TCAS indicates the airplane is clear of the conflict.

http://www.bfu-web.de/berichte/02_ax001dfr.pdf

EXAMPLE: IS $x/(x-y)$ WELL-DEFINED?

Many things may go wrong

- x and/or y may be uninitialized;
- $x-y$ may overflow;
- x and y may be equal (or $x-y$ may underflow): division by 0;
- $x/(x-y)$ may overflow (or underflow).

EXAMPLE: VALIDATION OF ARRAY REFERENCES

Are these array accesses safe?

```
procedure shellsort(n, t)
begin
  var n : integer = input integer;
  var a : array [0..n-1] of integer = input integer;

  var h : integer;
  var i : integer;
  var j : integer;
  var B : integer;

  if n >= 0 then begin
    h := 1;
    while (h*3 + 1) < n do
      h := 3*h + 1;
```

```
while h > 0 do begin
  i := h-1;
  while i < n do begin
    B := a[i];
    j := i;
    while (j >= h) and (a[j-h] > B) do begin
      a[j] := a[j-h];
      j := j-h
    end;
    a[j] := B;
    i := i+1
  end;
  h := h div 3
end
end
end
```

EXAMPLE: STRING CLEANNESS IN C/C++

Taken from Web2c: an implementation of TeX and friends that translates the original WEB sources into C. See, <http://www.tug.org/web2c/>.

```
#define BUFSIZ 1024
```

```
char buf[BUFSIZ];
```

```
char* insert_long(char *cp) {
```

```
    char temp[BUFSIZ];
```

```
    int i;
```

```
    assert(cp >= buf[0] && cp < buf[BUFSIZ]);
```

```
    for (i = 0; &buf[i] < cp; ++i)
```

```
        temp[i] = buf[i];
```

```
    strcpy (&temp[i], "(long)");
```

```
/* UNSAFE! */
```

```
    strcpy (&temp[i + 6], cp);
```

```
/* UNSAFE! */
```

```
    strcpy (buf, temp);
```

```
    return cp + 6;
```

```
/* UNSAFE! */
```

```
}
```

EXAMPLE: OVERFLOW OF SIGNED INTEGERS C/C++

Did you know that...

- ...in C/C++ a signed integer overflow results in **undefined behavior**?
- Not to be confused with **unspecified behavior**.
- “**Unspecified behavior**” means that each combination of architecture, operating system and compiler must consistently define what happens on overflow:
 - saturation?
 - wrap 2's (or 1's) complement?
 - exception?
 - termination?
- “**Undefined behavior**” means that anything can happen:
 - your hard disk may be formatted on Mondays;
 - a beep may be emitted on Fridays;
 - demons may fly out of your nose from time to time.

So.....

What can we do about it?

- full verification is undecidable;
- code review: complex, expensive and with volatile results;
- dynamic testing plus debugging: complex, expensive, does not scale (the cost of testing goes as the square of the program size), but it is repeatable;
- formal methods: complex and expensive but reusable, can be very thorough, repeatable, scale up to a certain program size then become unapplicable (we are working to extend that limit).

FORMAL PROGRAM VERIFICATION METHODS

Purpose

- To **mechanically prove** that **all** possible program executions are **correct** in all specified execution environments. . .
- . . . for some definition of **correct**:
 - absence of some kinds of run-time errors;
 - adherence to some partial specification. . .

Several methods

- deductive methods;
- model checking;
- program typing;
- **static analysis**.

Because of the undecidability of program verification

- all methods are partial or incomplete;
- all resort to some form of approximation.

ABSTRACT INTERPRETATION

- The right framework to work with the concept of **sound approximation**;
- a theory for approximating sets and set operations as considered in set (or category) theory, including inductive definitions;
- a theory of approximation of the behavior of dynamic discrete systems;
- Computation takes place on a domain of abstract properties: the **abstract domain**...
- ... using **abstract operations** which are sound approximations of the concrete operations.
- Correctness follows by design!
- The abstraction (approximation) can be coarse enough to be **finitely computable**, yet be precise enough to be practically useful.
- Examples: casting out of nines and rule of signs.

EXAMPLE: THE CONCRETE SEMANTICS

`x := 0; y := 0;`

`while x <= 100 do`

`$(x, y) \in S \in \wp(\mathbb{R}^2)$`

`read(b);`

`if b then x := x+2`

`else x := x+1; y := y+1;`

`endif`

`endwhile`

Concrete domain:

$$\langle \wp(\mathbb{R}^2), \subseteq, \emptyset, \mathbb{R}^2, \cup, \cap \rangle.$$

Concrete Semantics:

$$S \stackrel{\text{def}}{=} \text{lfp } \mathcal{F} = \mathcal{F}^\omega(\emptyset).$$

EXAMPLE: THE CONCRETE SEMANTICS

`x := 0; y := 0;`

`while x <= 100 do`

`$\emptyset$`

`read(b);`

`if b then x := x+2`

`else x := x+1; y := y+1;`

`endif`

`endwhile`

EXAMPLE: THE CONCRETE SEMANTICS

`x := 0; y := 0;`

`{(0,0)}`

`while x <= 100 do`

`$\emptyset$`

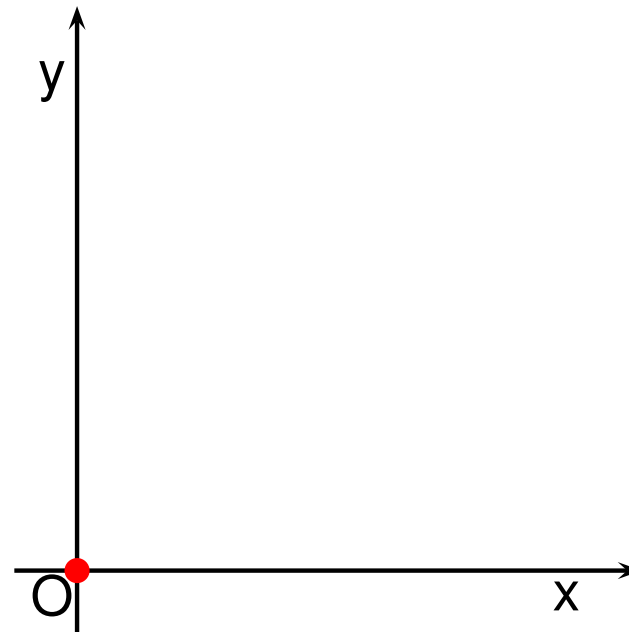
`read(b);`

`if b then x := x+2`

`else x := x+1; y := y+1;`

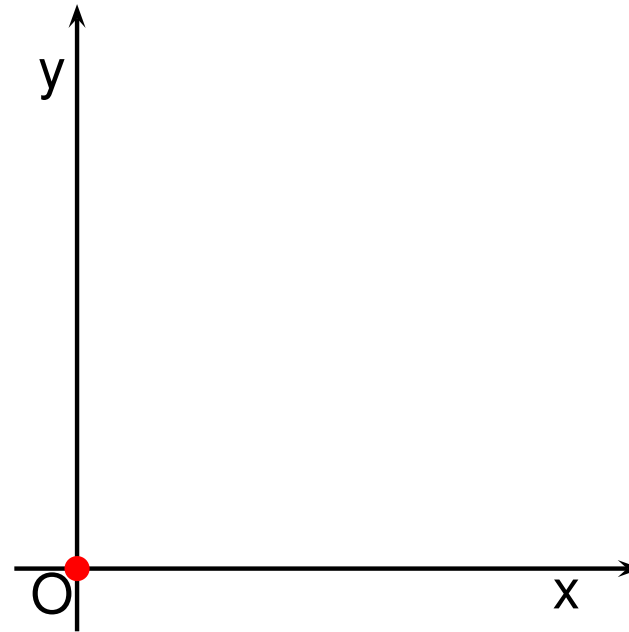
`endif`

`endwhile`



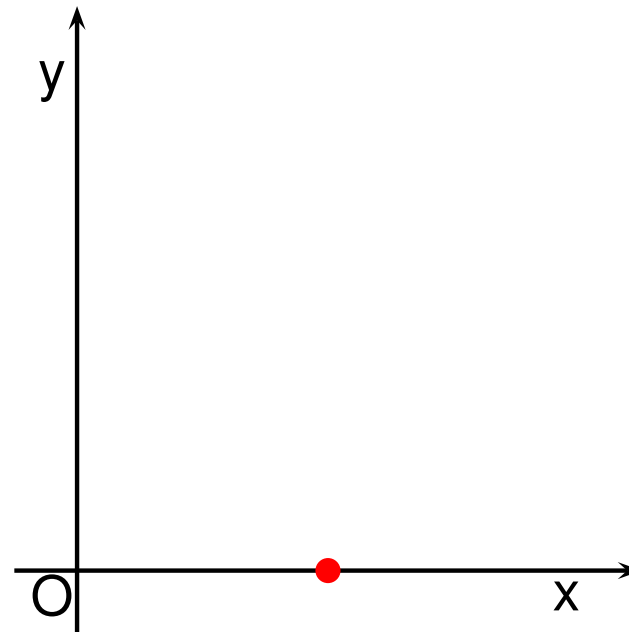
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0,0)}  
while x <= 100 do  
  {(0,0)}  
  read(b);  
  if b then x := x+2  
  
  else x := x+1; y := y+1;  
  
endif  
  
endwhile
```



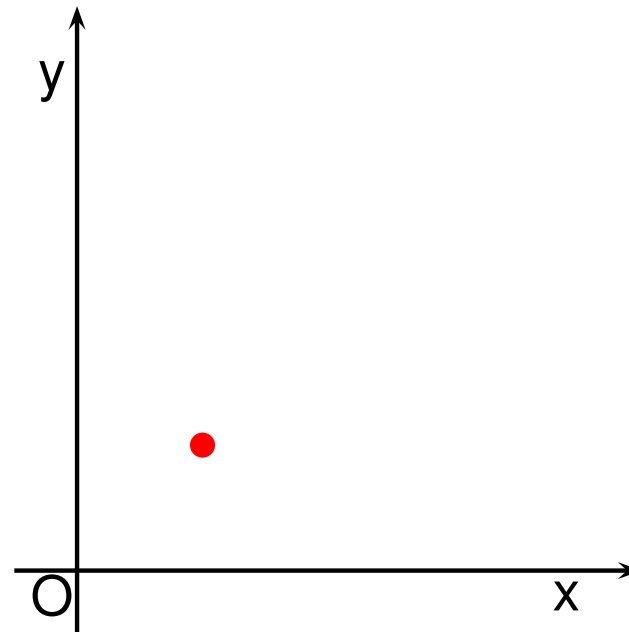
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0,0)}  
while x <= 100 do  
  {(0,0)}  
  read(b);  
  if b then x := x+2  
    {(2,0)}  
  else x := x+1; y := y+1;  
  
endif  
  
endwhile
```



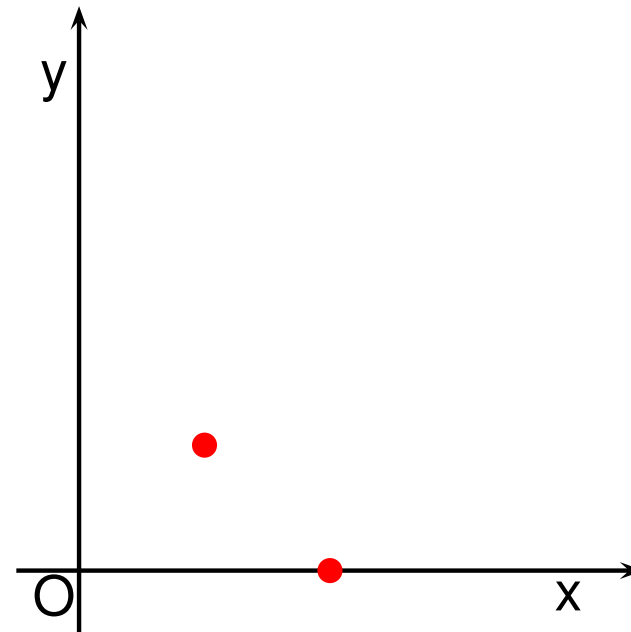
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0,0)}  
while x <= 100 do  
  {(0,0)}  
  read(b);  
  if b then x := x+2  
    {(2,0)}  
  else x := x+1; y := y+1;  
    {(1,1)}  
  endif  
  
endwhile
```



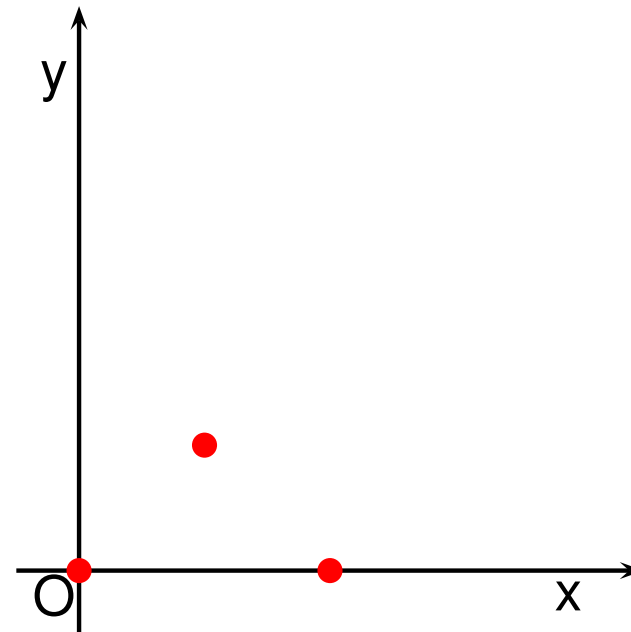
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0,0)}  
while x <= 100 do  
  {(0,0)}  
  read(b);  
  if b then x := x+2  
    {(2,0)}  
  else x := x+1; y := y+1;  
    {(1,1)}  
  endif  
  {(1,1), (2,0)}  
endwhile
```



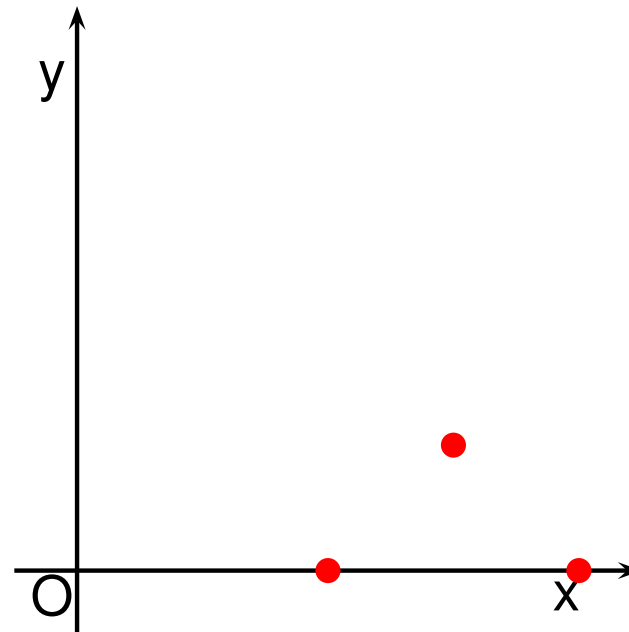
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0, 0)}  
while x <= 100 do  
  {(0, 0), (1, 1), (2, 0)}  
  read(b);  
  if b then x := x+2  
    {(2, 0)}  
  else x := x+1; y := y+1;  
    {(1, 1)}  
  endif  
  {(1, 1), (2, 0)}  
endwhile
```



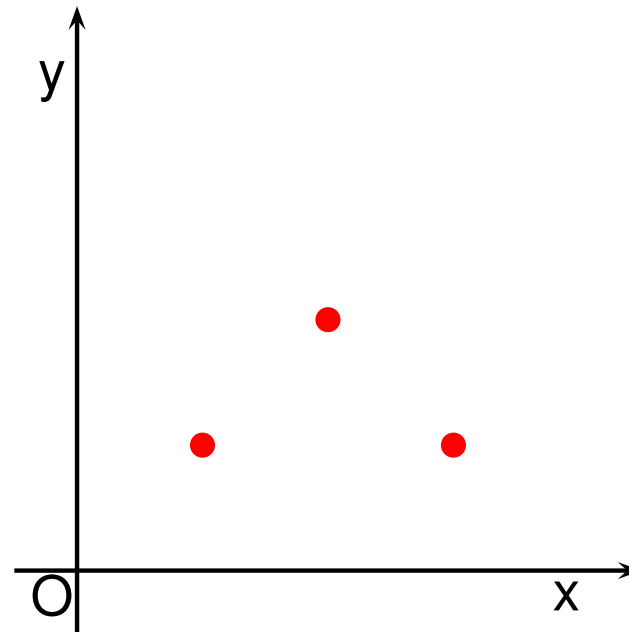
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0, 0)}  
while x <= 100 do  
  {(0, 0), (1, 1), (2, 0)}  
  read(b);  
  if b then x := x+2  
    {(2, 0), (3, 1), (4, 0)}  
  else x := x+1; y := y+1;  
    {(1, 1)}  
  endif  
  {(1, 1), (2, 0)}  
endwhile
```



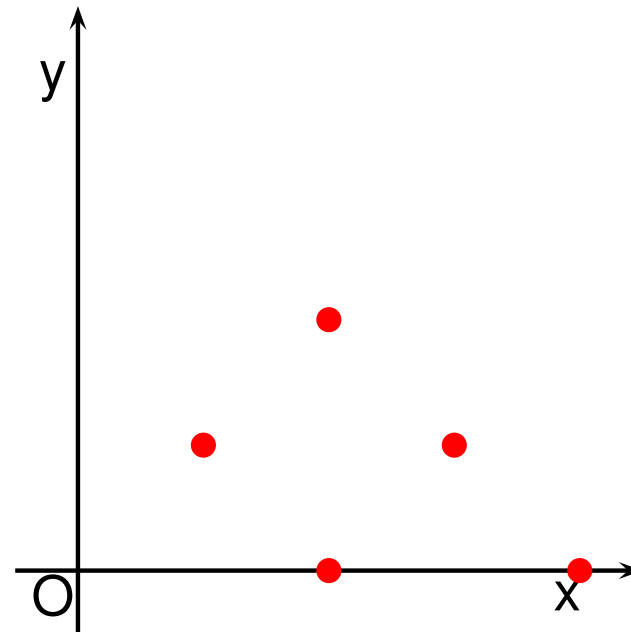
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0, 0)}  
while x <= 100 do  
  {(0, 0), (1, 1), (2, 0)}  
  read(b);  
  if b then x := x+2  
    {(2, 0), (3, 1), (4, 0)}  
  else x := x+1; y := y+1;  
    {(1, 1), (2, 2), (3, 1)}  
  endif  
  {(1, 1), (2, 0)}  
endwhile
```



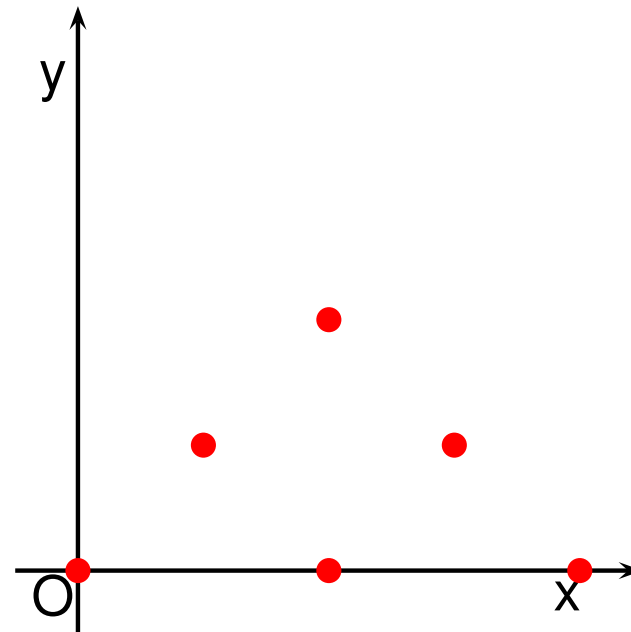
EXAMPLE: THE CONCRETE SEMANTICS

```
x := 0; y := 0;  
  {(0, 0)}  
while x <= 100 do  
  {(0, 0), (1, 1), (2, 0)}  
  read(b);  
  if b then x := x+2  
    {(2, 0), (3, 1), (4, 0)}  
  else x := x+1; y := y+1;  
    {(1, 1), (2, 2), (3, 1)}  
  endif  
  {(1, 1), (2, 0), (2, 2), (3, 1), (4, 0)}  
endwhile
```



EXAMPLE: ... AND SO ON ...

```
x := 0; y := 0;  
  {(0, 0)}  
while x <= 100 do  
  {(0, 0), (1, 1), (2, 0), (2, 2), (3, 1), (4, 0)}  
  read(b);  
  if b then x := x+2  
    {(2, 0), (3, 1), (4, 0)}  
  else x := x+1; y := y+1;  
    {(1, 1), (2, 2), (3, 1)}  
  endif  
  {(1, 1), (2, 0), (2, 2), (3, 1), (4, 0)}  
endwhile
```



EXAMPLE: THE ABSTRACT SEMANTICS

`x := 0; y := 0;`

`while x <= 100 do`

`$(x, y) \in \mathcal{Q} \in \mathbb{CP}_2$`

`read(b);`

`if b then x := x+2`

`else x := x+1; y := y+1;`

`endif`

`endwhile`

Abstract domain:

$$\langle \mathbb{CP}_2, \subseteq, \emptyset, \mathbb{R}^2, \uplus, \cap \rangle.$$

Correctness:

$$X \subseteq \mathcal{P} \implies \mathcal{F}(X) \subseteq \mathcal{F}^\#(\mathcal{P}).$$

Abstract Semantics:

$$\mathcal{Q} \in \text{postfp}(\mathcal{F}^\#).$$

EXAMPLE: THE ABSTRACT SEMANTICS

`x := 0; y := 0;`

`while x <= 100 do`

`{1 = 0}`

`read(b);`

`if b then x := x+2`

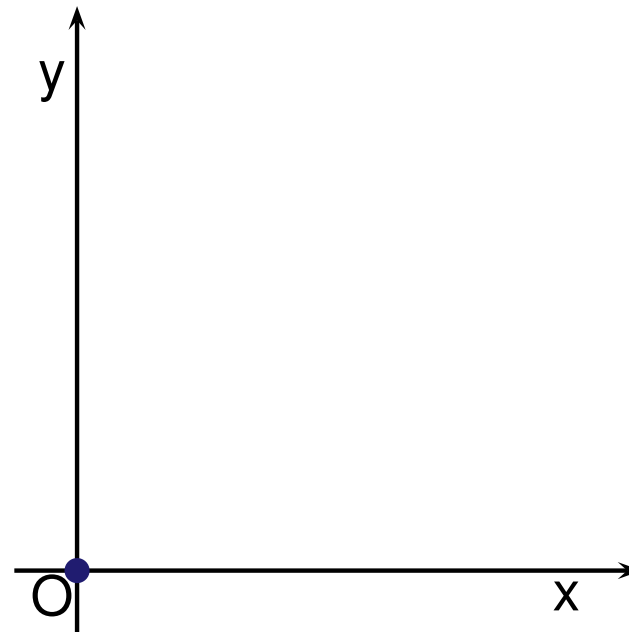
`else x := x+1; y := y+1;`

`endif`

`endwhile`

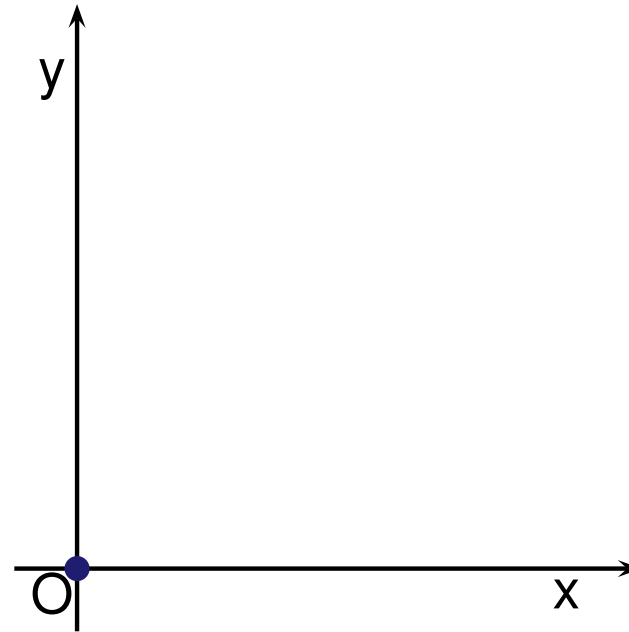
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $1 = 0$ }  
  read(b);  
  if b then x := x+2  
  
  else x := x+1; y := y+1;  
  
endif  
  
endwhile
```



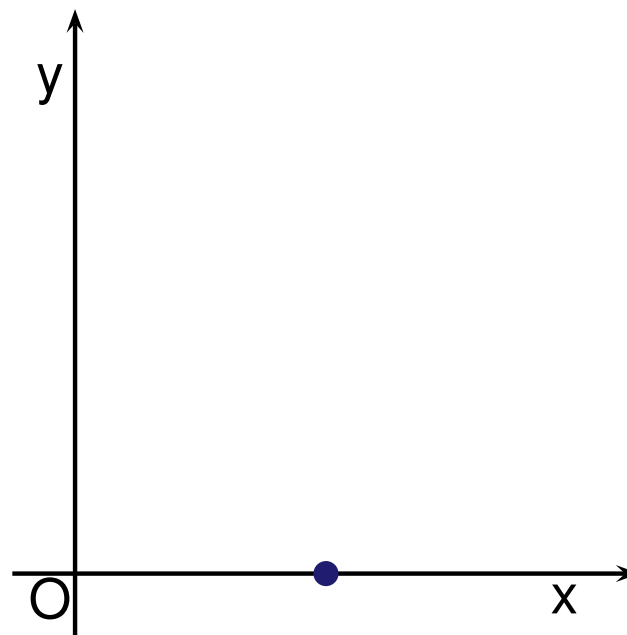
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $x = 0, y = 0$ }  
  read(b);  
  if b then x := x+2  
  
  else x := x+1; y := y+1;  
  
endif  
  
endwhile
```



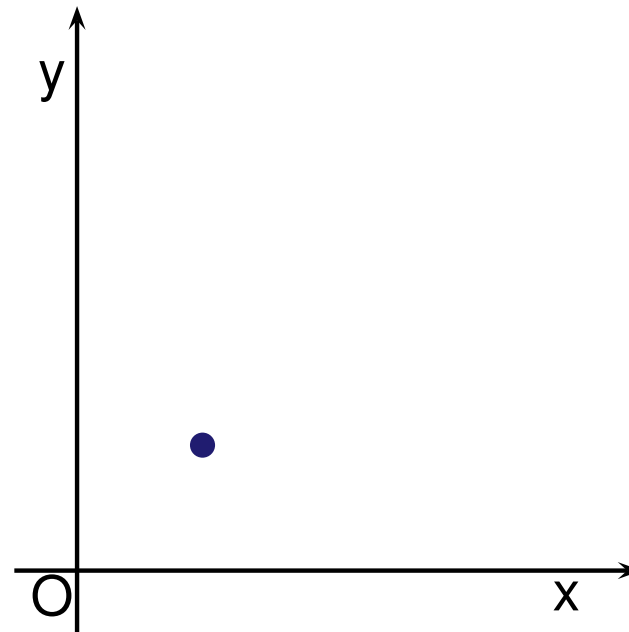
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $x = 0, y = 0$ }  
  read(b);  
  if b then x := x+2  
    { $x = 2, y = 0$ }  
  else x := x+1; y := y+1;  
  
endif  
  
endwhile
```



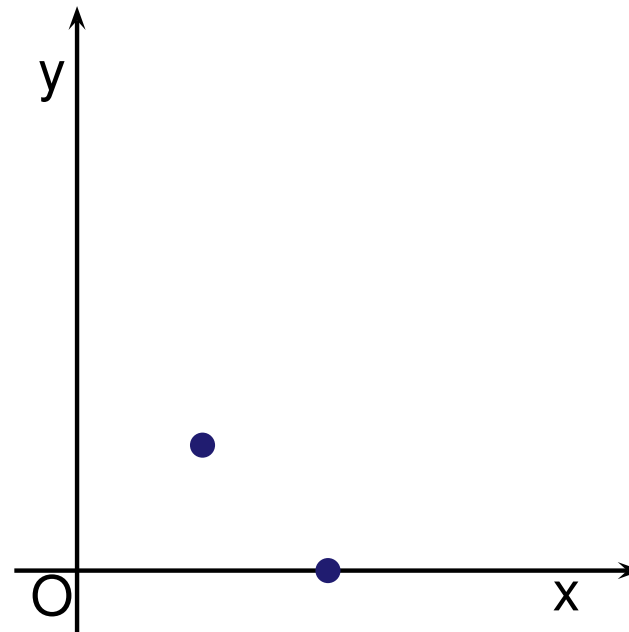
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $x = 0, y = 0$ }  
  read(b);  
  if b then x := x+2  
    { $x = 2, y = 0$ }  
  else x := x+1; y := y+1;  
    { $x = 1, y = 1$ }  
  endif  
endwhile
```



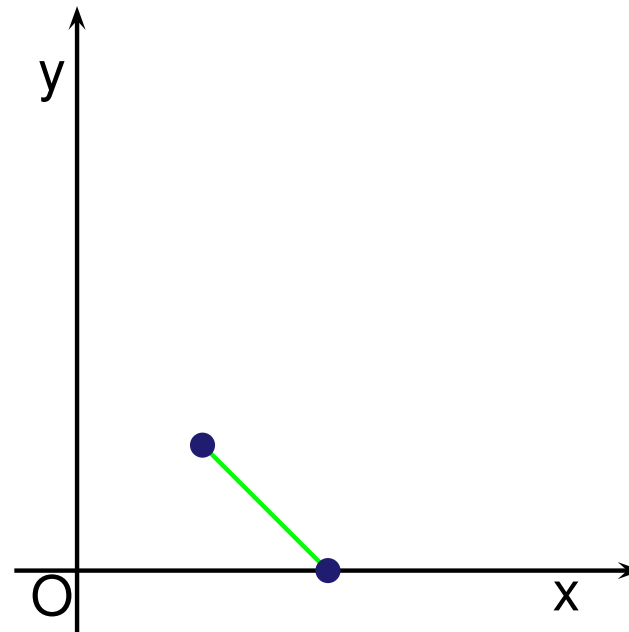
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $x = 0, y = 0$ }  
  read(b);  
  if b then x := x+2  
    { $x = 2, y = 0$ }  
  else x := x+1; y := y+1;  
    { $x = 1, y = 1$ }  
  endif  
  { $x = 2, y = 0$ }  $\uplus$  { $x = 1, y = 1$ }  
endwhile
```



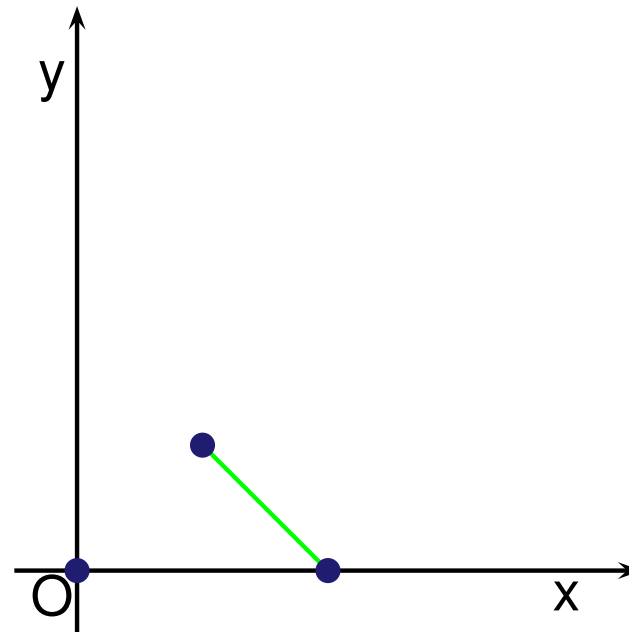
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $x = 0, y = 0$ }  
  read(b);  
  if b then x := x+2  
    { $x = 2, y = 0$ }  
  else x := x+1; y := y+1;  
    { $x = 1, y = 1$ }  
  endif  
  { $1 \leq x \leq 2, x + y = 2$ }  
endwhile
```



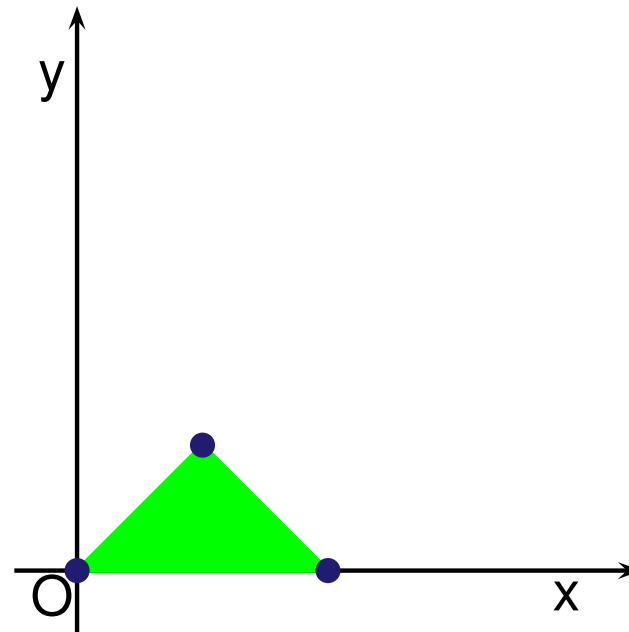
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $x = 0, y = 0$ }  
   $\uplus \{1 \leq x \leq 2, x + y = 2\}$   
  read(b);  
  if b then x := x+2  
    { $x = 2, y = 0$ }  
  else x := x+1; y := y+1;  
    { $x = 1, y = 1$ }  
  endif  
  { $1 \leq x \leq 2, x + y = 2$ }  
endwhile
```



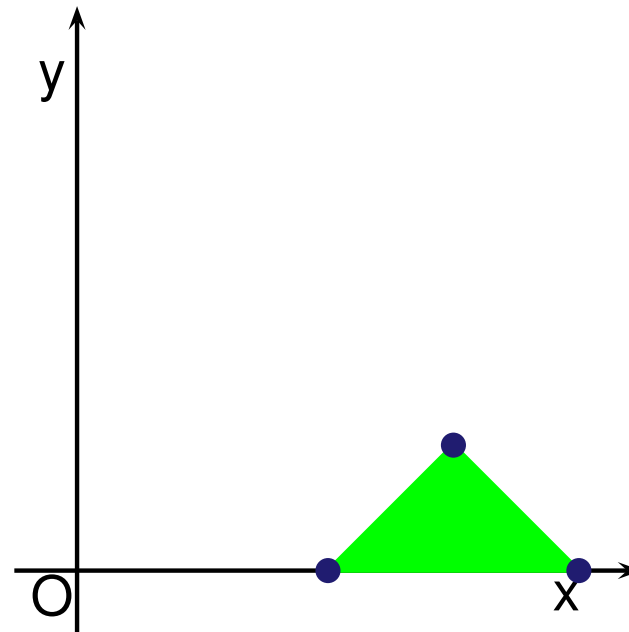
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 2$ }  
  read(b);  
  if b then x := x+2  
    { $x = 2, y = 0$ }  
  else x := x+1; y := y+1;  
    { $x = 1, y = 1$ }  
  endif  
  { $1 \leq x \leq 2, x + y = 2$ }  
endwhile
```



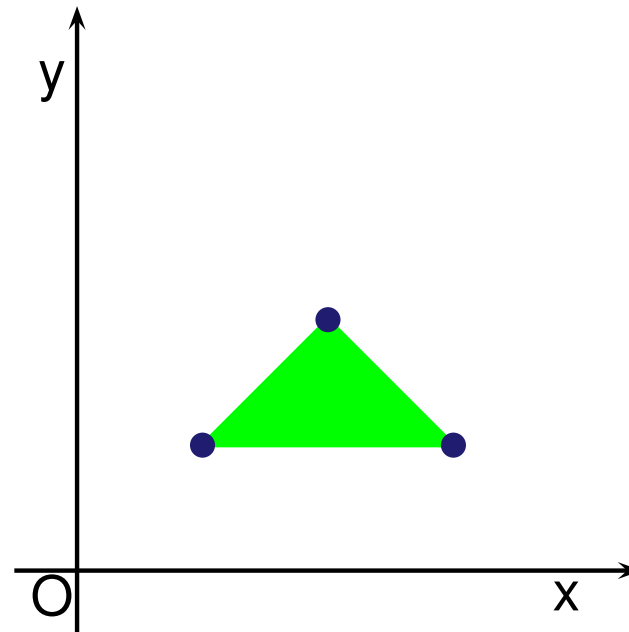
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 2$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $x = 1, y = 1$ }  
  endif  
  { $1 \leq x \leq 2, x + y = 2$ }  
endwhile
```



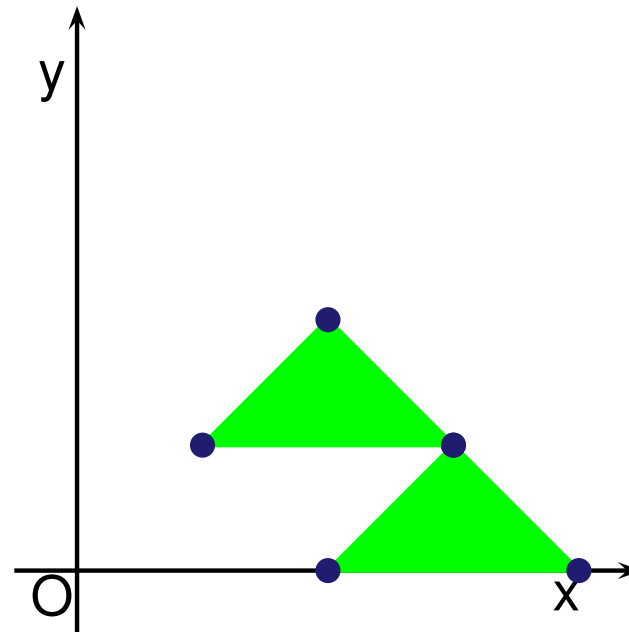
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 2$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $1 \leq x \leq 2, x + y = 2$ }  
endwhile
```



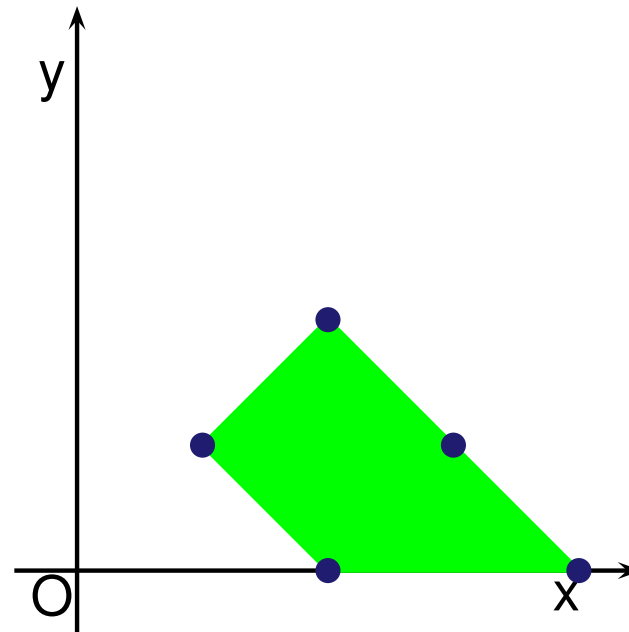
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 2$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $0 \leq y \leq x - 2, x + y \leq 4$ }  
     $\uplus$  { $1 \leq x \leq 2, x + y = 2$ }  
endwhile
```



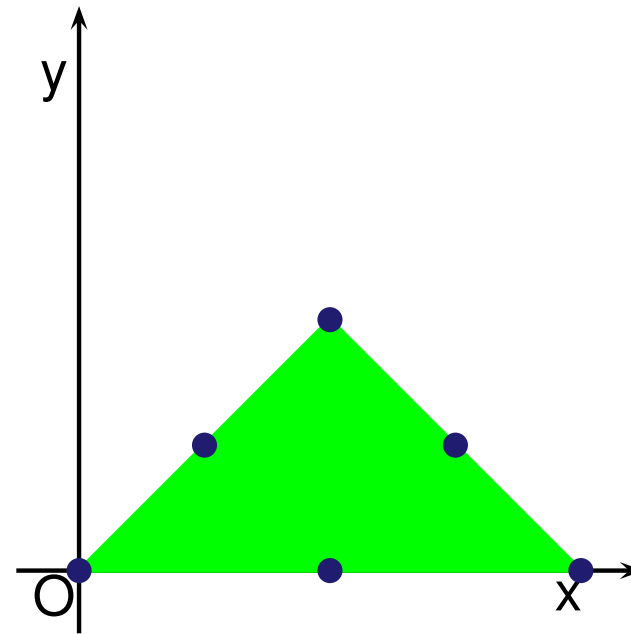
EXAMPLE: THE ABSTRACT SEMANTICS

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 2$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y \leq 4$ }  
endwhile
```



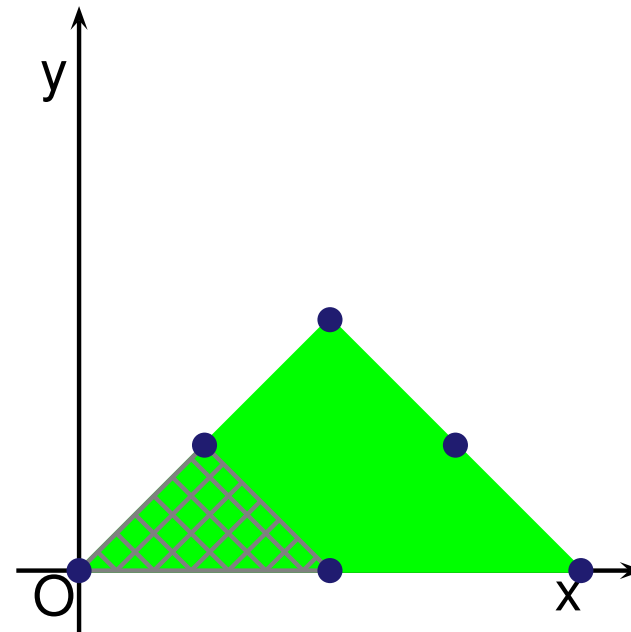
EXAMPLE: ... AND SO ON ... ?

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 4$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y \leq 4$ }  
endwhile
```



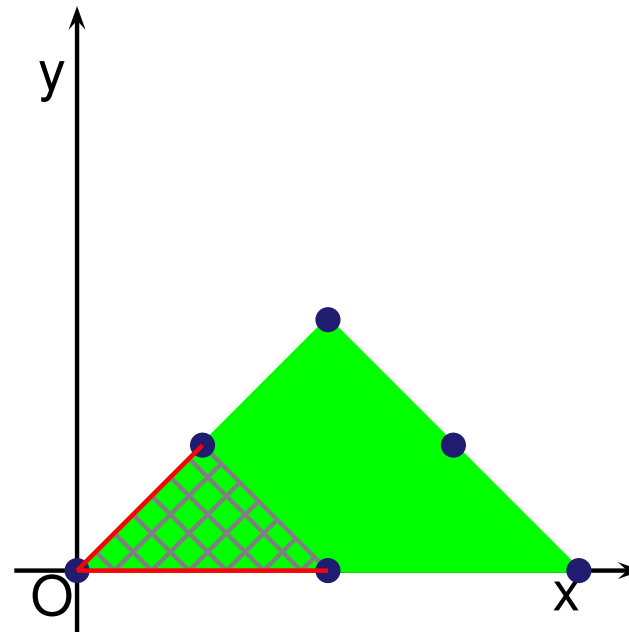
EXAMPLE: FINITE CONVERGENCE USING WIDENING

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 2$ }  
   $\nabla$  { $0 \leq y \leq x, x + y \leq 4$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y \leq 4$ }  
endwhile
```



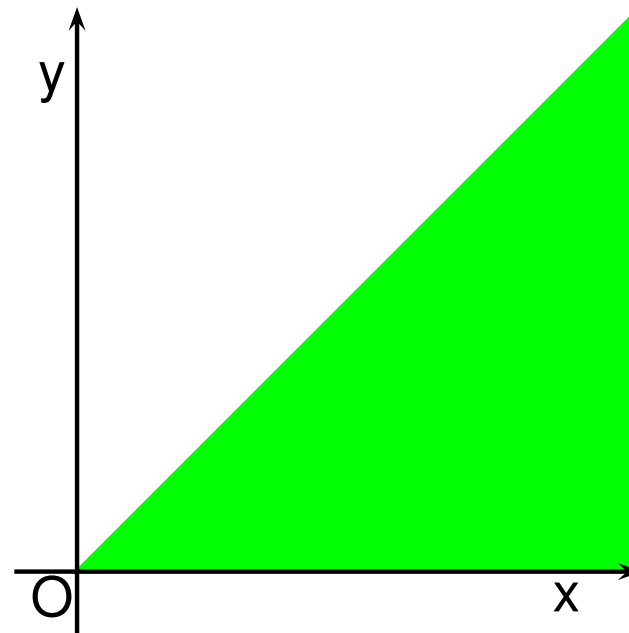
EXAMPLE: FINITE CONVERGENCE USING WIDENING

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x, x + y \leq 2$ }  
   $\nabla$  { $0 \leq y \leq x, x + y \leq 4$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y \leq 4$ }  
endwhile
```



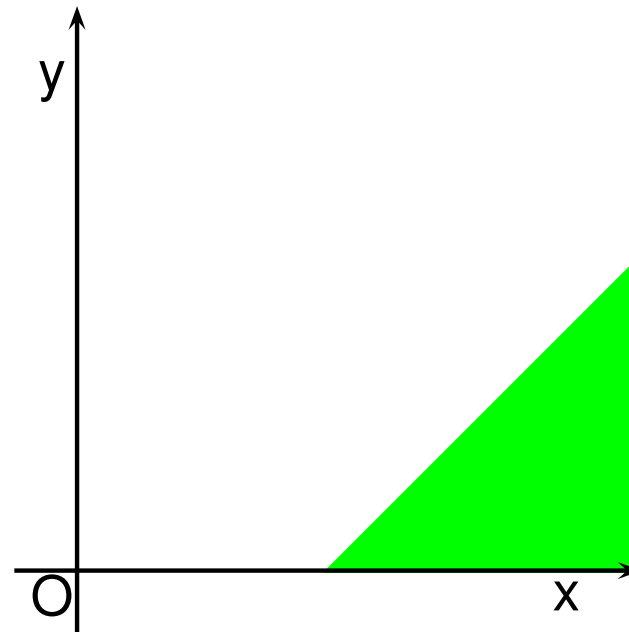
EXAMPLE: AN ABSTRACT POST-FIXPOINT

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2, x + y \leq 4$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y \leq 4$ }  
endwhile
```



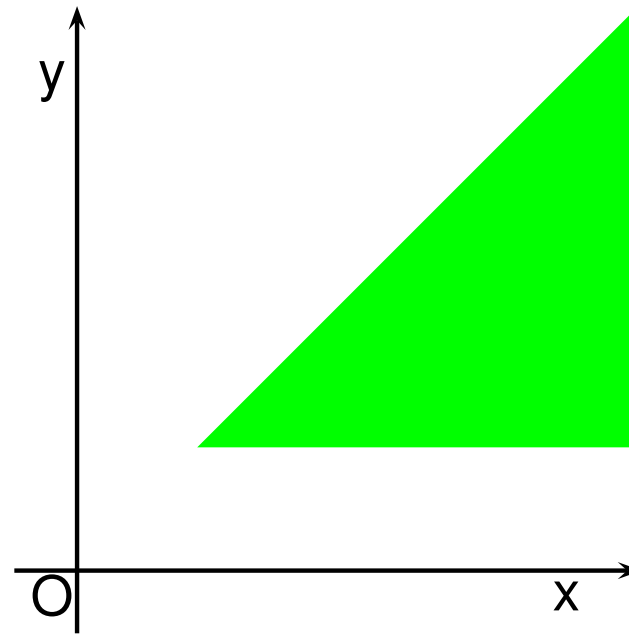
EXAMPLE: ABSTRACT DOWNWARD ITERATION

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x, x + y \leq 4$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y \leq 4$ }  
endwhile
```



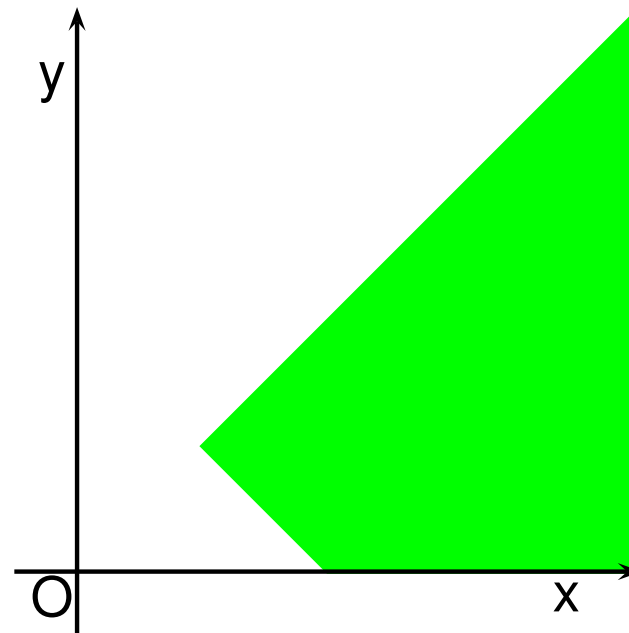
EXAMPLE: ABSTRACT DOWNWARD ITERATION

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y \leq 4$ }  
endwhile
```



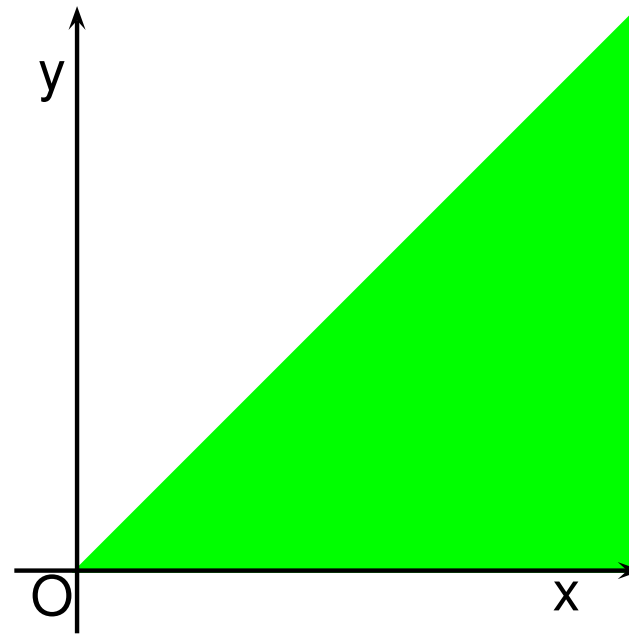
EXAMPLE: ABSTRACT DOWNWARD ITERATION

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y$ }  
endwhile
```



EXAMPLE: ABSTRACT FIXPOINT

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x \leq 100$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x$ }  
  endif  
  { $0 \leq y \leq x, 2 \leq x + y$ }  
endwhile
```



EXAMPLE: ABSTRACT FIXPOINT

```
x := 0; y := 0;  
  { $x = 0, y = 0$ }  
while x <= 100 do  
  { $0 \leq y \leq x \leq 100$ }  
  read(b);  
  if b then x := x+2  
    { $0 \leq y \leq x - 2 \leq 100$ }  
  else x := x+1; y := y+1;  
    { $1 \leq y \leq x \leq 101$ }  
  endif  
  { $0 \leq y \leq x \leq 102, 2 \leq x + y \leq 202$ }  
endwhile  
  { $100 < x \leq 102, 0 \leq y \leq x, x + y \leq 202$ }
```

THE (WELL-KNOWN) MORAL OF THE STORY

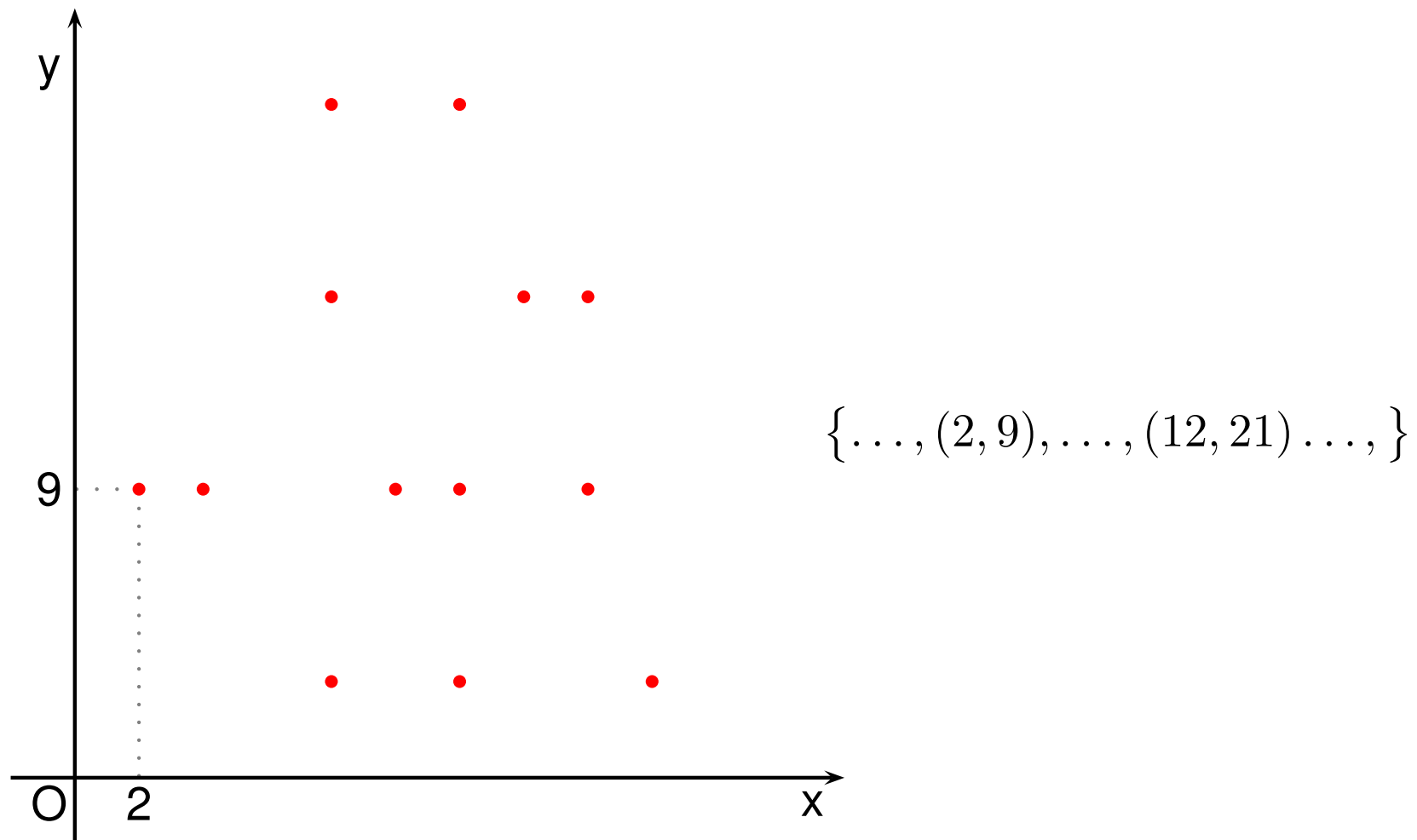
Semantic construction: language dependent, but (almost) independent from the specific application and, in particular, from the considered abstract domain.

Abstract Domain: semantic construction dependent, as far as the set of supported abstract operators is concerned. For some important cases (e.g., **numerical abstractions**) it is almost language and application independent.

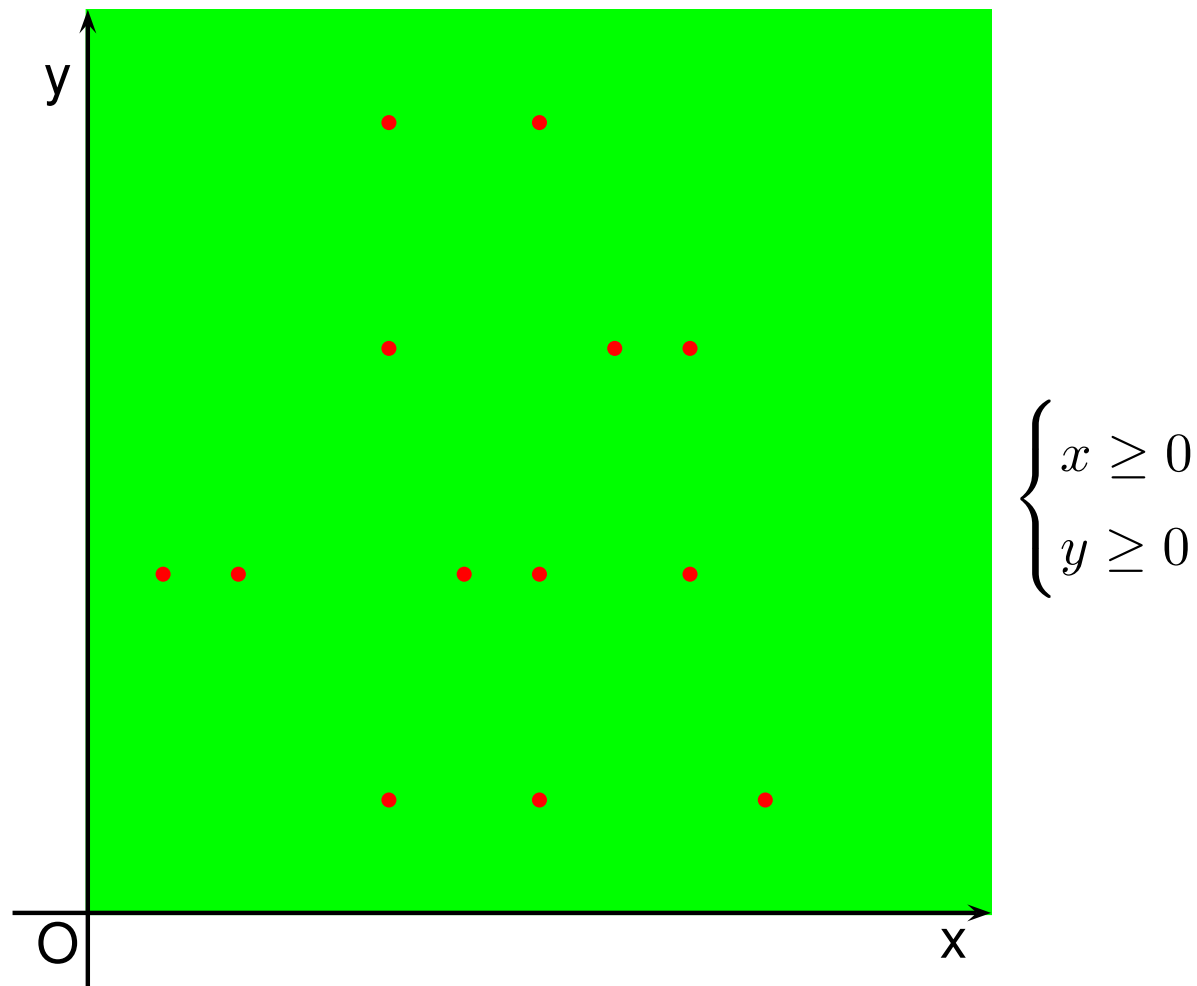
For a better understanding of **both** theoretical and practical research issues, the above separation of concerns should be pursued as much as possible.

We start by reviewing numerical abstract domains.

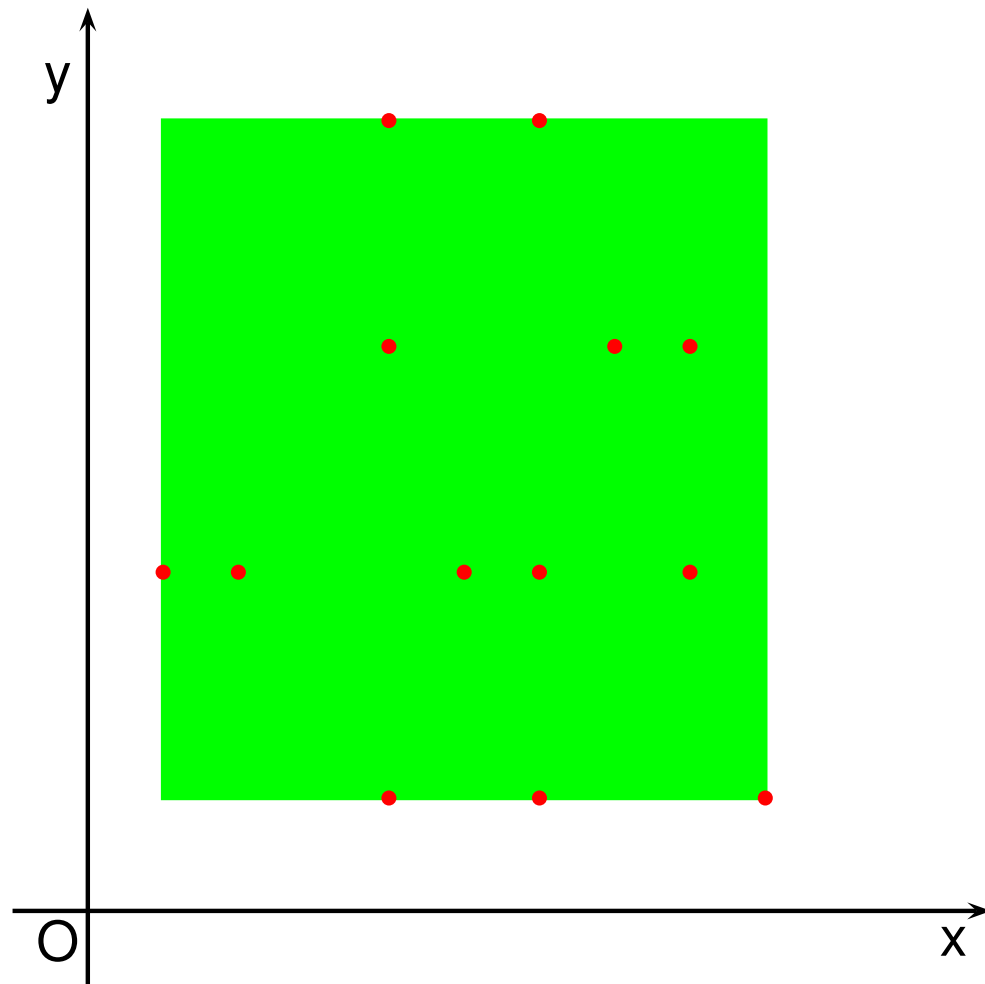
NUMERICAL ABSTRACTIONS: NO ABSTRACTION



NUMERICAL ABSTRACTIONS: SIGNS

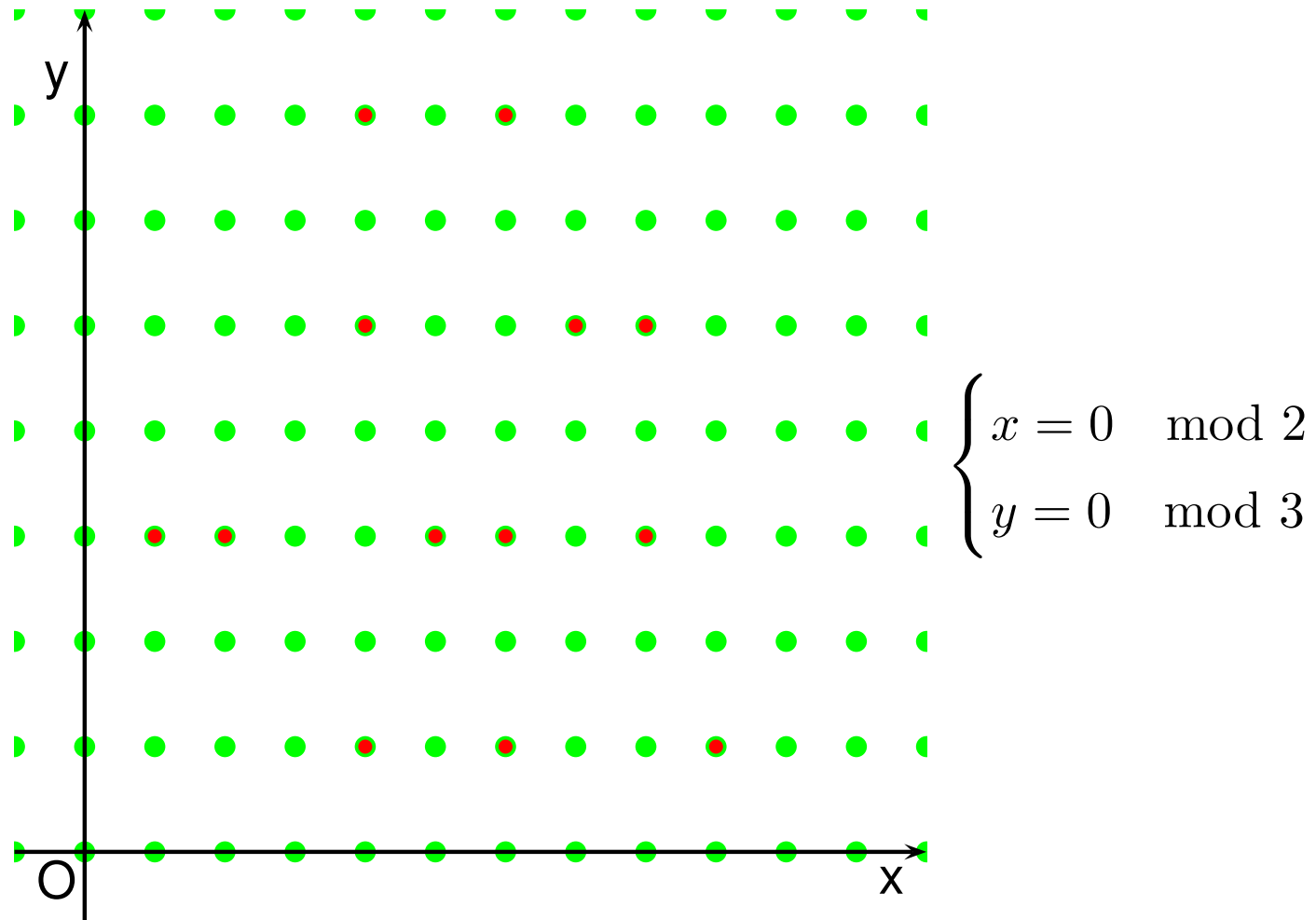


NUMERICAL ABSTRACTIONS: BOUNDING BOXES

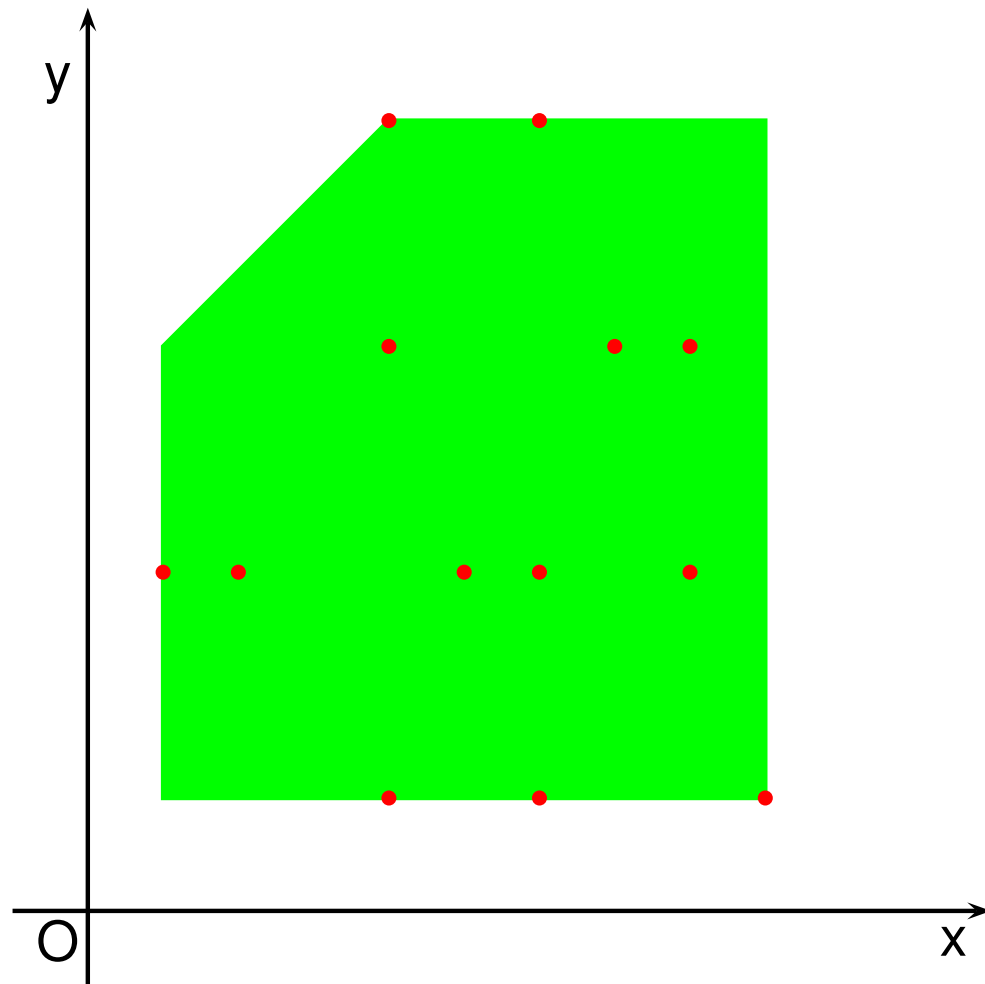


$$\begin{cases} 2 \leq x \leq 18 \\ 3 \leq y \leq 21 \end{cases}$$

NUMERICAL ABSTRACTIONS: SIMPLE CONGRUENCES

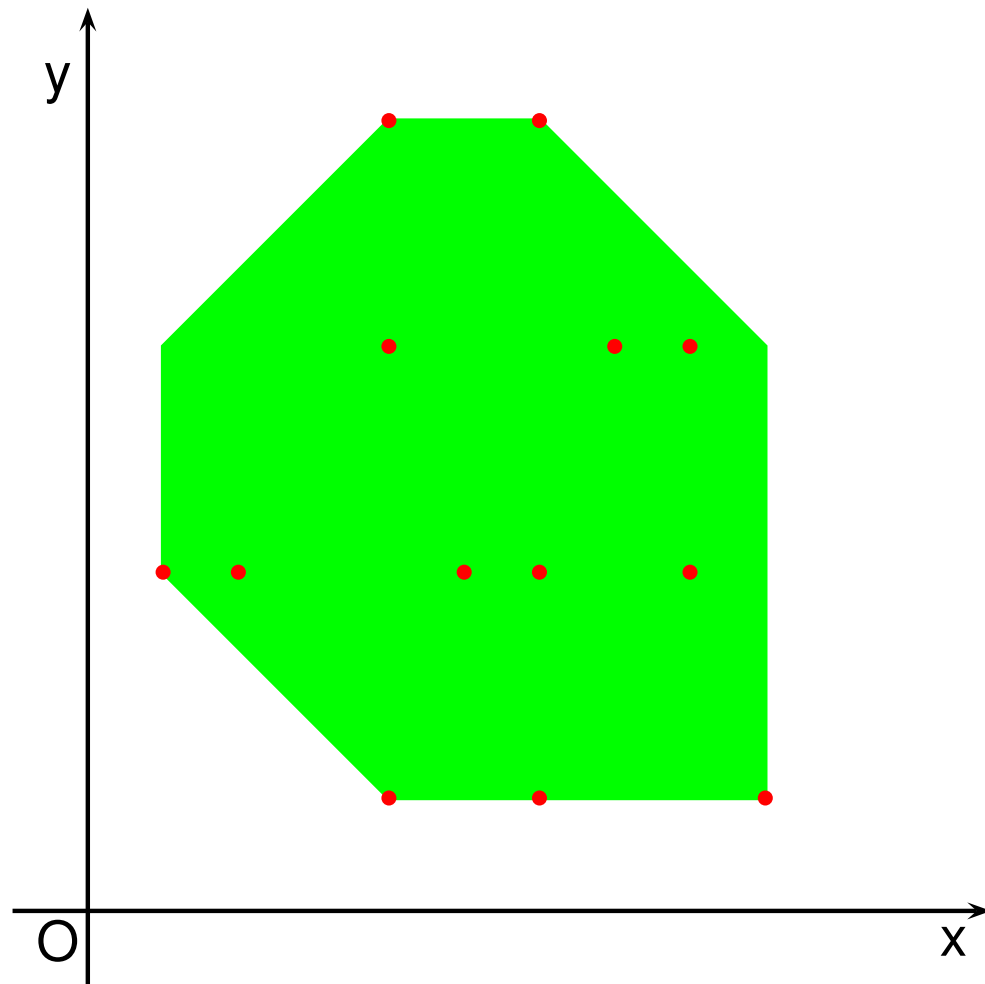


NUMERICAL ABSTRACTIONS: BOUNDED DIFFERENCES



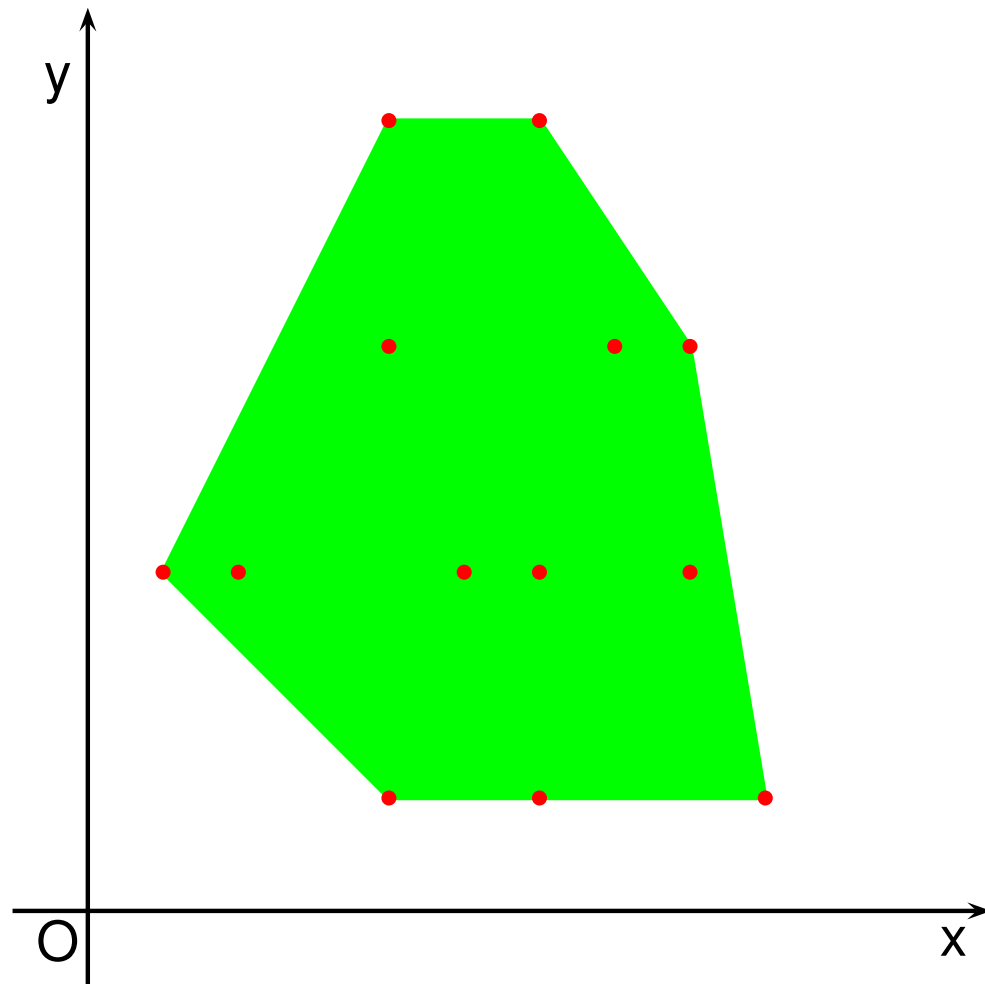
$$\begin{cases} 2 \leq x \leq 18 \\ 3 \leq y \leq 21 \\ -10 \leq x - y \end{cases}$$

NUMERICAL ABSTRACTIONS: OCTAGONS



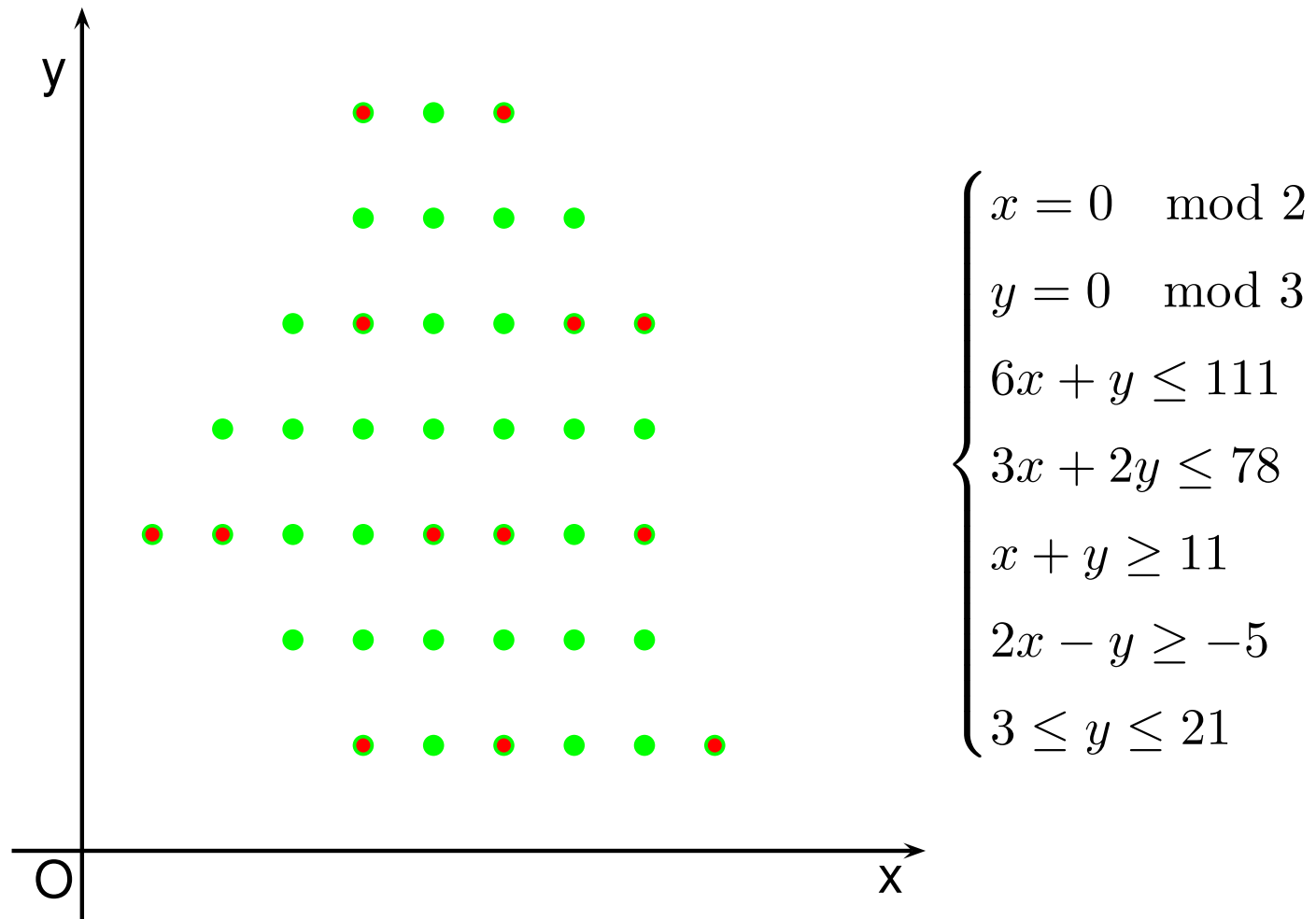
$$\begin{cases} 2 \leq x \leq 18 \\ 3 \leq y \leq 21 \\ -10 \leq x - y \\ 11 \leq x + y \leq 33 \end{cases}$$

NUMERICAL ABSTRACTIONS: CONVEX POLYHEDRA

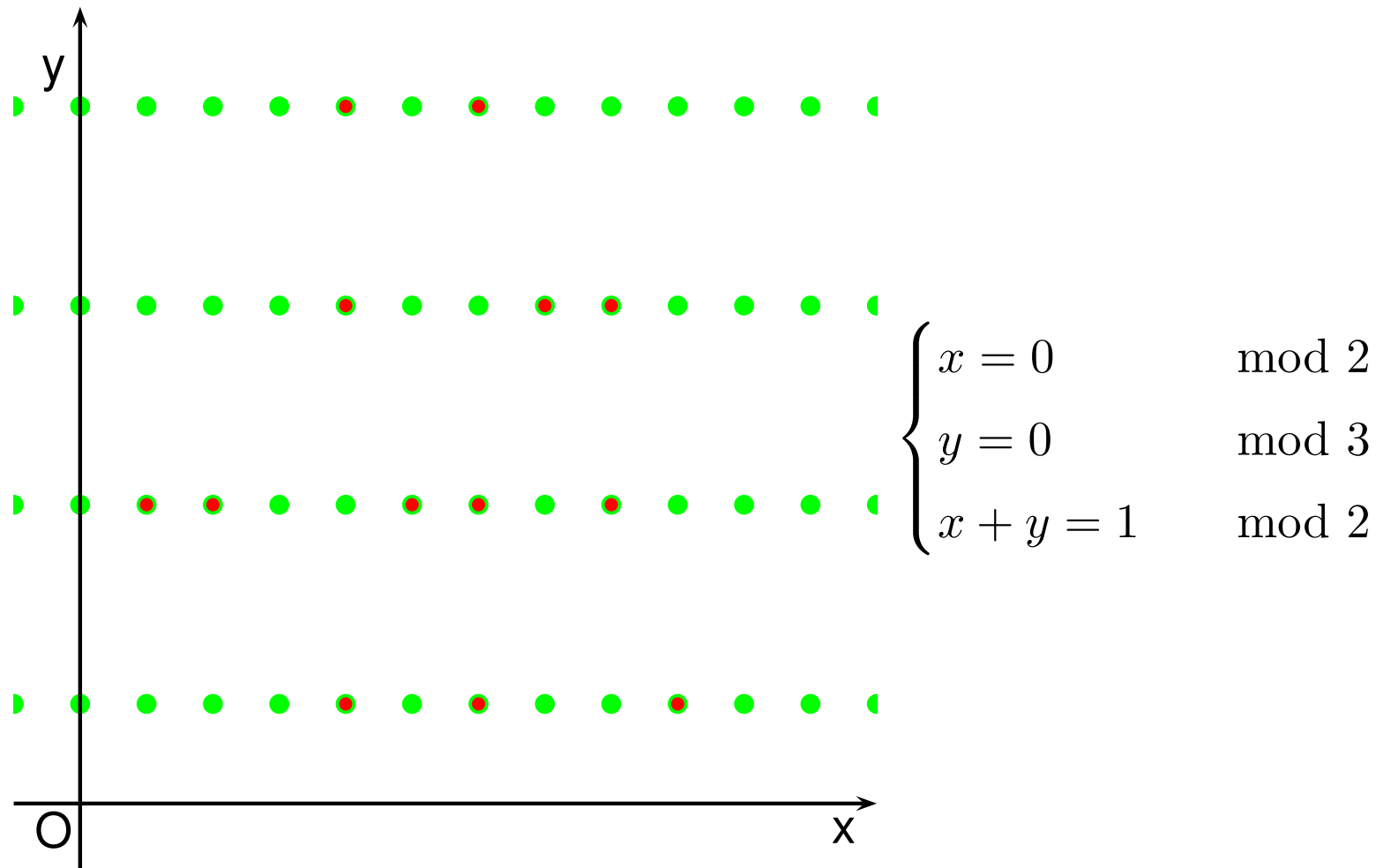


$$\begin{cases} 6x + y \leq 111 \\ 3x + 2y \leq 78 \\ x + y \geq 11 \\ 2x - y \geq -5 \\ 3 \leq y \leq 21 \end{cases}$$

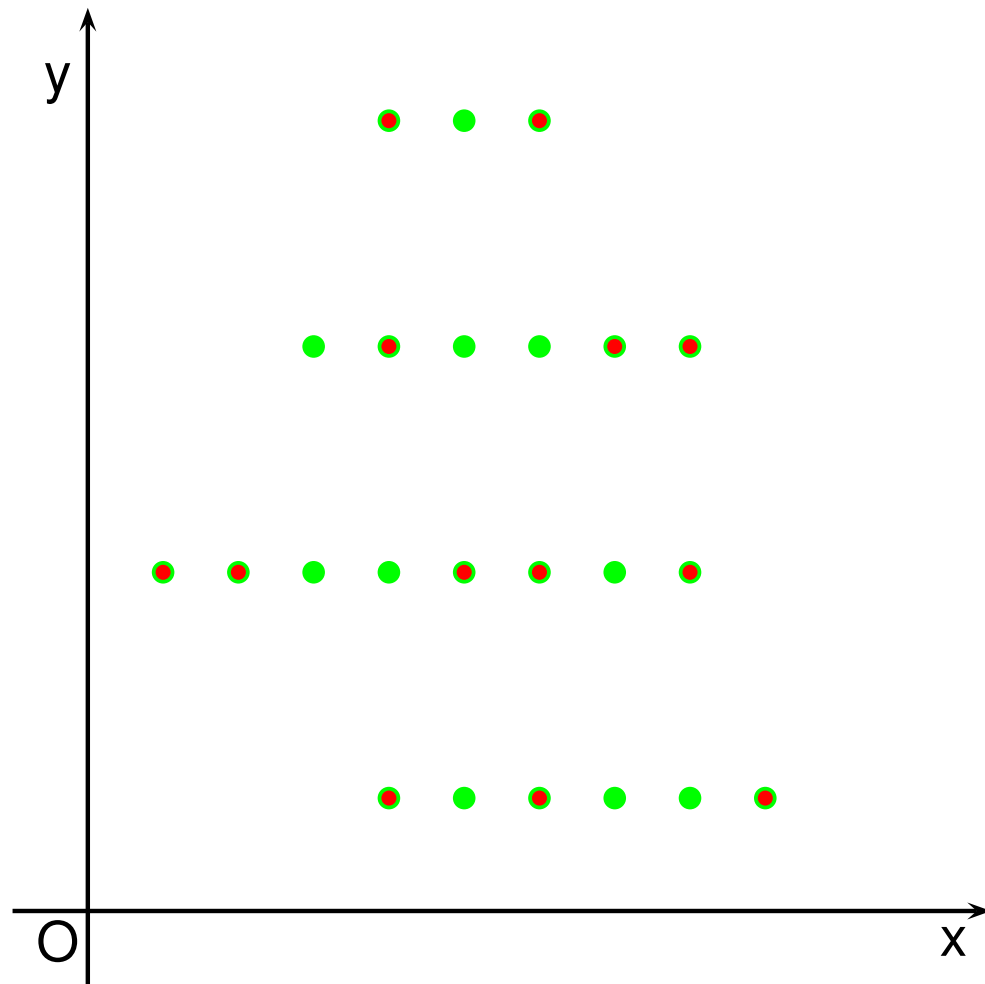
NUMERICAL ABSTRACTIONS: $\mathbb{Z}$ -POLYHEDRA (I)



NUMERICAL ABSTRACTIONS: RELATIONAL CONGRUENCES

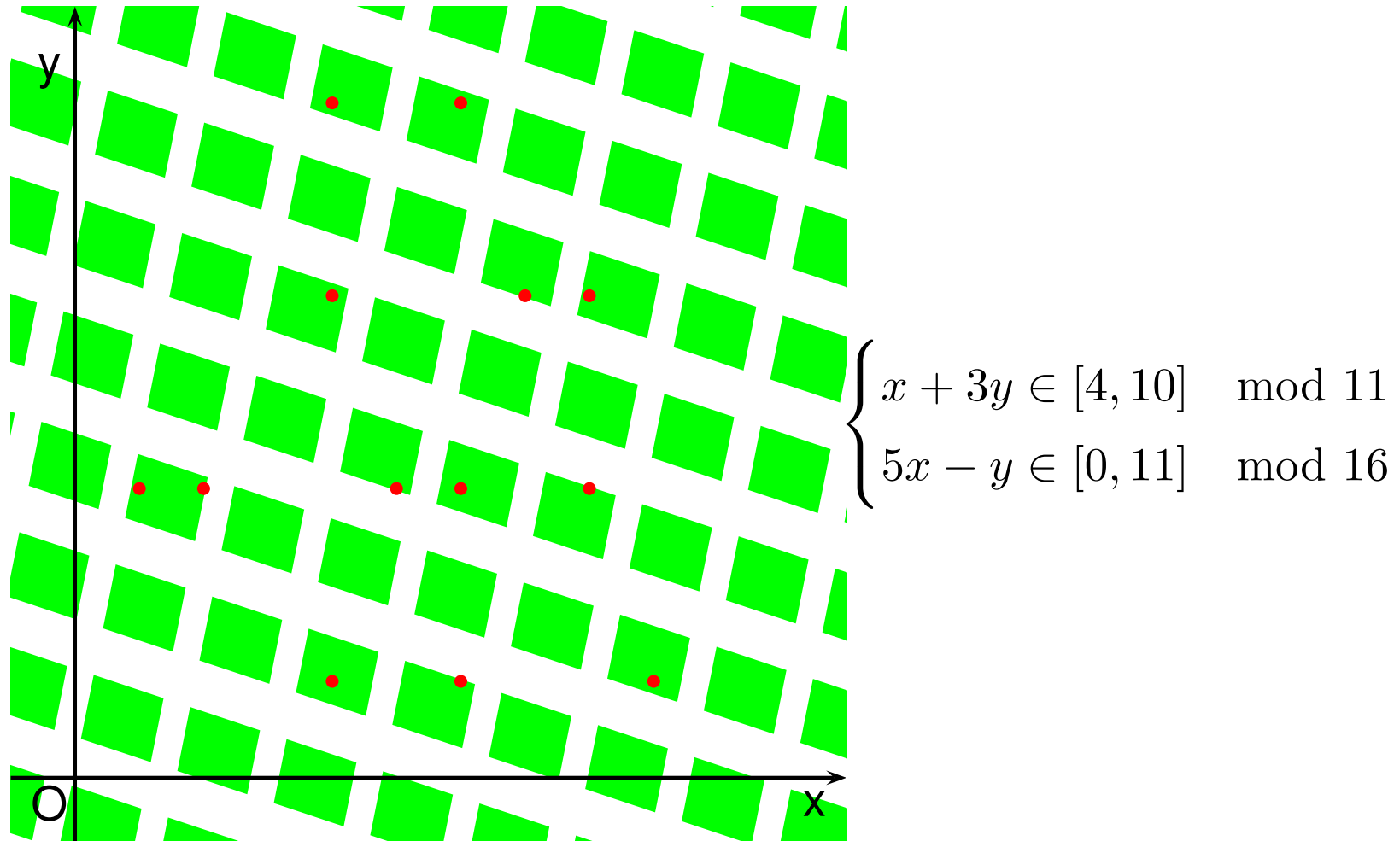


NUMERICAL ABSTRACTIONS: $\mathbb{Z}$ -POLYHEDRA (II)

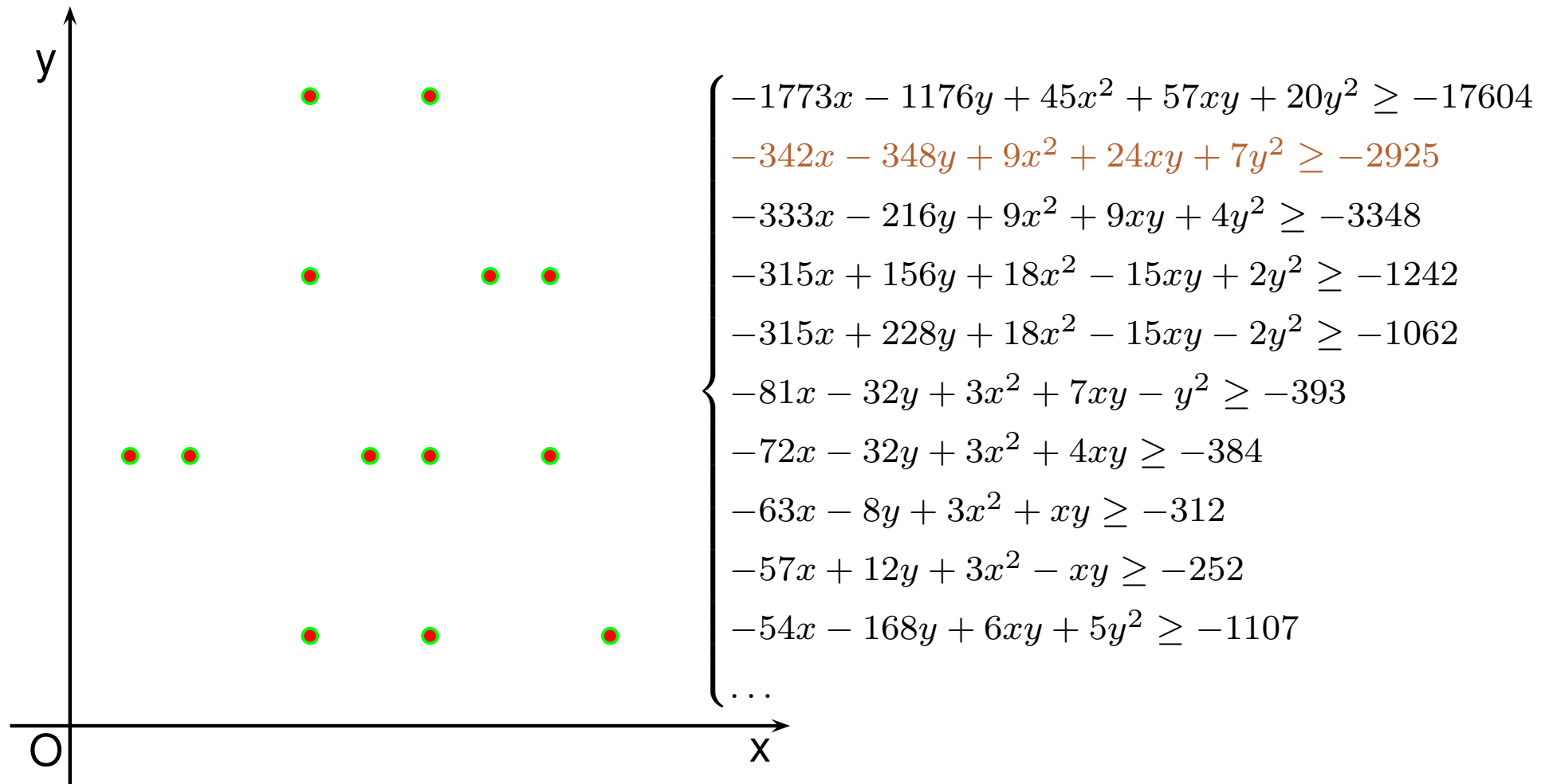


$$\left\{ \begin{array}{l} x = 0 \pmod{2} \\ y = 0 \pmod{3} \\ x + y = 1 \pmod{2} \\ 6x + y \leq 111 \\ 3x + 2y \leq 78 \\ x + y \geq 11 \\ 2x - y \geq -5 \\ 3 \leq y \leq 21 \end{array} \right.$$

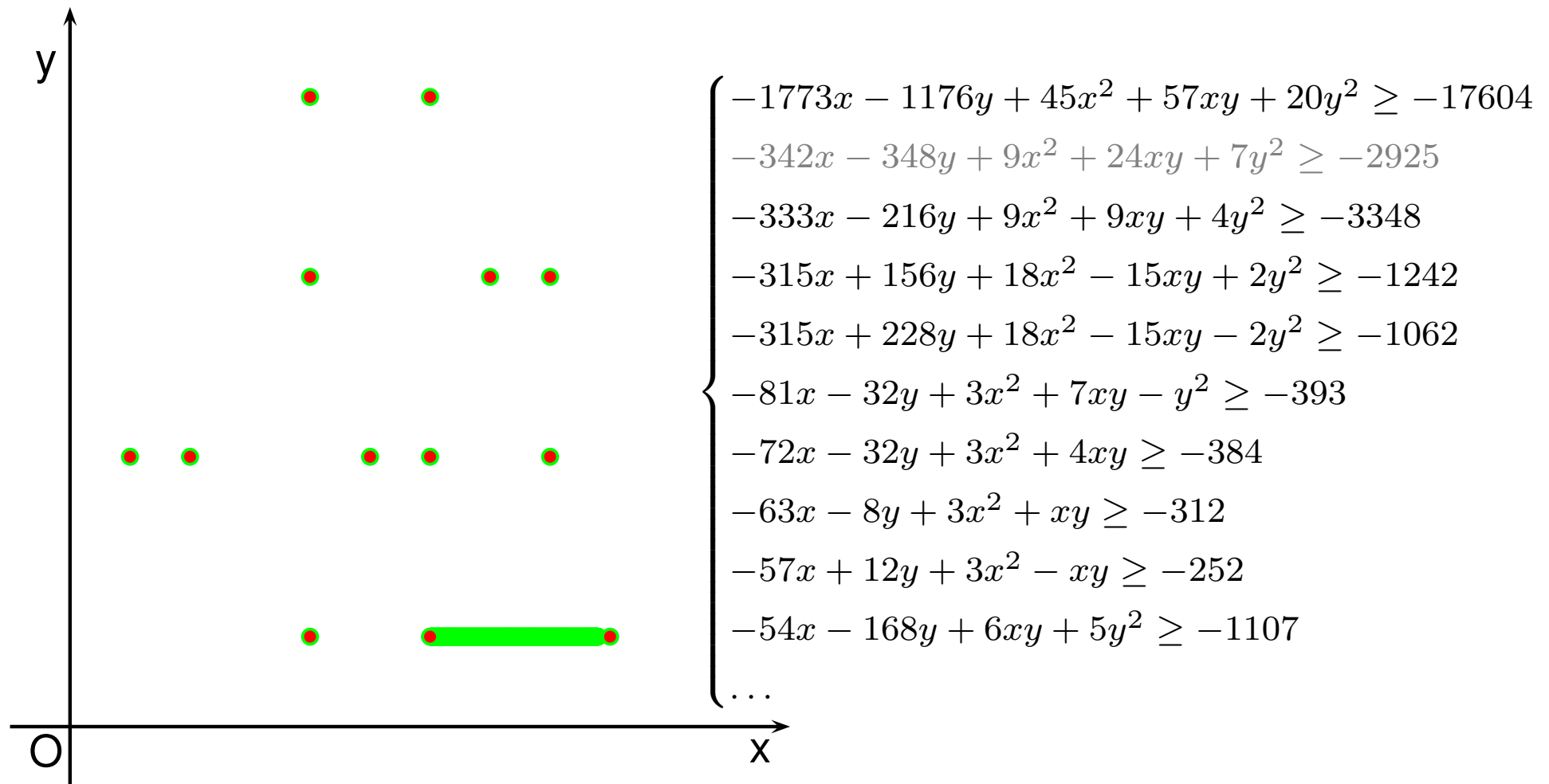
NUMERICAL ABSTRACTIONS: TRAPEZOIDAL CONGRUENCES



NUMERICAL ABSTRACTIONS: POLYNOMIAL CONES (I)



NUMERICAL ABSTRACTIONS: POLYNOMIAL CONES (II)



A TASTE OF ONE CONCRETE SEMANTICS FOR IMP (I)

Arithmetic Operations: Unexceptional Cases Denote by $\boxdot$ denotes any abstract syntax operator in $\{+, -, *, \mathbf{div}, \mathbf{mod}\}$ and by ' $\circ$ ' the corresponding arithmetic operation in $\{+, -, *, \div, \text{mod}\}$:

$$\frac{\rho \vdash \langle a_0, \sigma \rangle \rightarrow \xi}{\rho \vdash \langle a_0 \boxdot a_1, \sigma \rangle \rightarrow \xi}$$
$$\frac{\rho \vdash \langle a_0, \sigma \rangle \rightarrow m_0 \quad \rho \vdash \langle a_1, \sigma \rangle \rightarrow \xi}{\rho \vdash \langle a_0 \boxdot a_1, \sigma \rangle \rightarrow \xi}$$
$$\frac{\rho \vdash \langle a_0, \sigma \rangle \rightarrow m_0 \quad \rho \vdash \langle a_1, \sigma \rangle \rightarrow m_1}{\rho \vdash \langle a_0 \boxdot a_1, \sigma \rangle \rightarrow m_0 \circ m_1} \quad \text{if } \boxdot \notin \{\mathbf{div}, \mathbf{mod}\} \text{ or } m_1 \neq 0$$

A TASTE OF ONE CONCRETE SEMANTICS FOR IMP (II)

Division and Modulus: Exceptional Case Let $\boxdot \in \{\mathbf{div}, \mathbf{mod}\}$. The rules governing the behavior in case of division by zero are defined by the schema

$$\frac{\rho \vdash \langle a_0, \sigma \rangle \rightarrow m_0 \quad \rho \vdash \langle a_1, \sigma \rangle \rightarrow 0}{\rho \vdash \langle a_0 \boxdot a_1, \sigma \rangle \rightarrow \mathbf{divbyzero}}$$

A TASTE OF ONE ABSTRACT SEMANTICS FOR IMP

Division and Modulus Let $\boxdot \in \{\mathbf{div}, \mathbf{mod}\}$ and $\odot \in \{\oslash, \oplus\}$ denote the corresponding abstract operation:

$$\frac{\rho^\# \vdash \langle a_0, \sigma^\# \rangle \rightarrow \langle m_0^\#, \xi_0^\# \rangle \quad \rho^\# \vdash \langle a_1, \sigma^\# \rangle \rightarrow \langle m_1^\#, \xi_1^\# \rangle}{\rho^\# \vdash \langle a_0 \boxdot a_1, \sigma^\# \rangle \rightarrow \langle m_0^\# \odot m_1^\#, \xi_0^\# \sqcup \xi_1^\# \rangle} \quad \text{if } 0 \not\propto m_1^\#$$
$$\frac{\rho^\# \vdash \langle a_0, \sigma^\# \rangle \rightarrow \langle m_0^\#, \xi_0^\# \rangle \quad \rho^\# \vdash \langle a_1, \sigma^\# \rangle \rightarrow \langle m_1^\#, \xi_1^\# \rangle}{\rho^\# \vdash \langle a_0 \boxdot a_1, \sigma^\# \rangle \rightarrow \langle m_0^\# \odot m_1^\#, \xi_0^\# \sqcup \xi_1^\# \sqcup \alpha(\{\mathbf{divbyzero}\}) \rangle} \quad \text{if } 0 \propto m_1^\#$$